

01: Role Map and Study Plan

This chapter describes the role these study guides prepare for, the stages of the interview process, and a four-week plan that assigns each guide in this directory to a week. Read it first, and use the plan to decide which guides to study and in what order.

This chapter covers

- The responsibilities of a senior cloud software engineer on a GPU cloud team
- What each interview stage tests, and how to prepare for it
- A four-week study plan mapped to the guides and code in this repository
- A short checklist of material to prepare before the onsite interviews

Contents

The role	2
Interview stages	3
1. Recruiter screen	3
2. Technical phone screen	3
3. Virtual onsite	3
4. Team and hiring manager round	4
Four-week study plan	5
Preparation checklist	6

The role

The target role is **Senior Cloud Software Engineer, Cloud GPU Services**. Engineers in this role build and operate the software that lets customers request, run, and monitor GPU workloads. The work covers four areas:

Area	What it includes
GPU cloud control plane	The APIs and services that accept requests, enforce quotas, and track workload state
Multi-tenant scheduling	Placing workloads on GPU capacity while isolating tenants from each other
Cluster lifecycle	Creating, upgrading, and repairing Kubernetes clusters and GPU node pools
Developer experience	The SDKs and command-line tools that customers use to call those APIs

Job descriptions for this role typically ask for the following skills. The right column lists the guide that covers each one.

Skill	Covered in
Building SDKs and CLIs for cloud services	07-cli-sdk-rest-concurrency.md, 12-rust-guide-part2-errors-concurrency-sdk.md
RESTful API design	06-system-design-cloud-gpu.md, 07-cli-sdk-rest-concurrency.md
Python, Go, or Rust	11, 12, and 13 (Rust guides), 04-kubernetes-operator.md (Go)
Tested, maintainable software, CI/CD, and automation	08-testing-cicd-observability.md
Performance tuning	02-linux-concepts.md, 14-concurrency-async-networking.md
Linux, containers, Kubernetes, and cloud platforms	02-linux-concepts.md, 03-kubernetes-concepts.md
Cross-team collaboration and documentation	09-behavioral-interview.md
Code generation and open-source experience (preferred)	10-gpu-operator-kubebuilder-deep-dive.md

Interview stages

The process usually has four stages. Each one tests something different, so prepare for them separately.

1. Recruiter screen

The recruiter confirms your background, level, technology stack, and compensation expectations. Prepare a one-sentence summary of your experience that matches the role, for example:

"I build cloud-facing APIs, SDKs, and CLIs, and the controllers behind them."

2. Technical phone screen

This is a coding interview of 45 to 60 minutes with one or two medium or hard problems. Python, Go, and Rust are usually accepted; the code in this repository is in Rust. Problems for this role often involve systems themes rather than pure algorithms:

Theme	Implementation in <code>rust-interview-lab/src/problems/</code>
An LRU cache	<code>lru_cache.rs</code> , <code>lru_cache_easy.rs</code>
A rate limiter or token bucket	<code>rate_limiter.rs</code>
A worker pool	<code>worker_pool.rs</code>
Retries with backoff	<code>retry.rs</code>
A shared counter across threads	<code>threads.rs</code> (<code>AtomicCounter</code>)
Parsing an HTTP request	<code>http_request.rs</code>
A minimal async executor	<code>async_mini.rs</code>
Ordering tasks by their dependencies	<code>graph_topology.rs</code>

Asynchronous HTTP clients and event-processing pipelines also come up. The lab does not implement them, but `14-concurrency-async-networking.md` covers the underlying mechanisms.

3. Virtual onsite

The onsite consists of several sessions, each about an hour:

Session	Typical content	Guides
System design	Design a GPU cloud service or a multi-tenant GPU cluster API	06
Linux and operating systems	Processes, memory, cgroups, namespaces, networking, debugging, performance tools	02, 14
Kubernetes	Control plane, scheduling, operators, CRDs, device plugins, Helm, the GPU Operator	03, 04, 10
Behavioral and leadership	Ownership, conflict, ambiguity, influence	09
Lab (sometimes)	A take-home or pair-programming exercise, such as building a small controller	05

4. Team and hiring manager round

The hiring manager assesses how you set priorities, handle ambiguous requirements, take ownership of problems, and work with other teams. Prepare examples from your own work using the structure in 09-behavioral-interview.md.

Four-week study plan

Each week has a focus area and a concrete result that shows the week is complete.

Week	Focus	Study	Done when
1	Linux and operating system fundamentals	02-linux-concepts.md	You have run every command and program in the guide and can walk through each debugging scenario without notes
2	Kubernetes and operator theory	03-kubernetes-concepts.md, 04-kubernetes-operator.md, 10-gpu-operator-kubebuilder-deep-dive.md	You can explain the path from <code>kubectl apply</code> to a running container, and the reconcile loop
3	Operator lab and system design	05-kubernetes-operator-lab.md, 06-system-design-cloud-gpu.md	<code>scripts/verify-kind.sh</code> passes, you can explain each line of the reconciler, and you have presented the design exercise aloud in 45 minutes
4	SDKs, CLIs, Rust, and behavioral practice	07, 08, 09, 11, 12, 13, 14, and rust-interview-lab/	You have solved the lab problems without looking at the solutions and completed at least two mock interviews

Preparation checklist

Prepare the following before the onsite interviews:

- **A project story.** One page describing the most complex cloud feature you delivered: the problem, your design, the trade-offs, what went wrong, and the result.
- **Behavioral examples in STAR form.** Situation, task, action, and result, for each question type in 09-behavioral-interview.md.
- **Measured results.** State outcomes with numbers, for example: "Reduced API p99 latency by 40% by adding client-side caching and asynchronous pagination."
- **A control plane diagram.** Practice drawing the path from the API server through the controllers and the device plugin to the GPU driver.
- **GPU component names.** When a question goes beyond what you know, name the components involved (driver, container toolkit, device plugin, MIG, DCGM, Triton) and explain what you do know about how they connect. Do not guess at details.

Source: [01-role-map-and-study-plan.md](#) · [PDF](#) · [All study guides](#)